

# Failure (a real rollback trigger) — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — The idea, and the exact pieces

### Basic Version

A rollback trigger watches recent test scores as they come in, and decides: "has quality dropped enough, for long enough, that we should go back to the last version we know was good?" A single bad score isn't enough on its own — you're watching the *trend* of the last several scores, not any one of them.

Things to use:

- A list that keeps growing as new scores come in.
- List slicing, `scores[-5:]`, to grab just the most recent few.
- A simple average: `sum(...) / len(...)`.
- An `if` comparing that average to a starting ("baseline") score.

### Intermediate Version

The shape of this exercise is: keep a running list of scores, and after every new one arrives, look at just the last `window_size` of them (a "sliding window") and compare their average against a baseline — the score you'd expect a healthy version to keep hitting. If the recent average has fallen far enough below the baseline — say, dropped by 20% or more — that's the trigger firing.

Percent drop is `(baseline - avg_recent) / baseline`. A baseline of `0.95` and a recent average of `0.60` gives `(0.95 - 0.60) / 0.95 ≈ 0.37`, a 37% drop — well past a `0.2` (20%) threshold.

The exact pieces:

- `scores_so_far[-window_size:]` — Python's negative-index slicing grabs the last `window_size` items of a list, however many there are (it won't error even if the list is shorter than `window_size`, which matters for the next point).
- A guard for not enough data yet: `if len(scores_so_far) < window_size: return False` — you can't trust a 2-score average to represent "the last 5 requests" if only 2 have happened.
- `percent_drop = (baseline - avg_recent) / baseline`, then `percent_drop >= drop_threshold`.
- A `rollback_to_previous()` function — for this exercise, a stub that just prints what it would do and returns the version name it reverted to; in the Build Task, this is the real function that rewrites the live pointer.

**Difference between Basic and Intermediate:** Basic names the sliding-window average and the drop threshold. Intermediate adds two log levels (quiet terminal, detailed file) and explains why the trigger keeps firing on every later request — and how one flag fixes that.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

### Basic Version

```
function should_rollback(scores_so_far, baseline, window_size,
                        drop_threshold) -> bool:
    if fewer than window_size scores so far: return False
    recent = the last window_size scores
    avg_recent = average of recent
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold
```

feed a list of simulated scores one at a time into should\_rollback  
the moment it returns True, call rollback\_to\_previous()  
and print when it happened

Here's almost the whole thing — just try running it and reading it line by line:

```
# rollback_trigger_practice.py
def should_rollback(scores_so_far, baseline, window_size=5, drop_threshold=0.2):
    if len(scores_so_far) < window_size:
        return False
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

def rollback_to_previous():
    print("ROLLBACK: reverting to last known-good version")
    return "v1"

scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
seen = []
for i, s in enumerate(scores):
    seen.append(s)
    if should_rollback(seen, baseline=0.95):
        print(f"request {i}: score {s} -- triggering rollback")
        rollback_to_previous()
```

**Expected output:** rollback fires (and keeps firing) once the recent-window average drops far enough — run it and find exactly which request number it first triggers on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

```
function should_rollback(scores_so_far: list[float], baseline: float,
                        window_size: int,
                        drop_threshold: float) -> bool:
    if not enough scores yet: return False
    recent = last window_size scores
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

function rollback_to_previous() -> str:
    print what happened
    return the version name rolled back to
```

```
# rollback_trigger_practice.py
def should_rollback(
    scores_so_far: list[float],
    baseline: float,
    window_size: int = 5,
    drop_threshold: float = 0.2,
) -> bool:
    if len(scores_so_far) < window_size:
        return False
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

def rollback_to_previous() -> str:
    print("ROLLBACK: reverting to last known-good version")
    return "v1"
```

Now write the loop that feeds a simulated sequence of scores through `should_rollback`, and notice it keeps calling `rollback_to_previous()` on every request after the first trigger — the repeat-firing problem Hint 1's Intermediate section named. Compare against the [Solution](#) once you've seen it happen yourself.

**Difference between Basic and Intermediate:** Basic checks the window and rolls back. Intermediate separates the watch loop into a function, records every window check at `DEBUG`, and

adds a flag so one decline causes exactly one rollback.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)